```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SIZE 10
#define HEAP_SIZE 50
#define MAX_QUEUE 50
#define MAX_MENU 20

// ================= FOOD STRUCT =================
struct Food {
    int id;
    char name[50];
    float price;
};

// ================= ORDER =================
struct Order {
    struct Food food;
    int qty;
};

// ================= BST =================
struct BST {
    struct Food food;
    struct BST *left, *right;
};

struct BST* createNode(struct Food f) {
    struct BST* node = (struct BST*)malloc(sizeof(struct BST));
    node->food = f;
    node->left = node->right = NULL;
    return node;
}

struct BST* insertBST(struct BST* root, struct Food f) {
    if(root == NULL) return createNode(f);

    if(f.id < root->food.id)
        root->left = insertBST(root->left, f);
    else
        root->right = insertBST(root->right, f);

    return root;
```

```
}

struct Food searchBST(struct BST* root, int id) {
    if(root == NULL) return (struct Food){-1,"",0};

    if(root->food.id == id)
        return root->food;

    if(id < root->food.id)
        return searchBST(root->left, id);

    return searchBST(root->right, id);
}

// ================= HASH TABLE =================
struct Food* hashTable[SIZE];

int hashFunction(int id) {
    return id % SIZE;
}

void insertHash(struct Food f) {
    int index = hashFunction(f.id);
    hashTable[index] = (struct Food*)malloc(sizeof(struct Food));
    *hashTable[index] = f;
}

struct Food searchHash(int id) {
    int index = hashFunction(id);

    if(hashTable[index] != NULL && hashTable[index]->id == id)
        return *hashTable[index];

    return (struct Food){-1,"",0};
}

// ================= QUEUE =================
struct Order queue[MAX_QUEUE];
int front = 0, rear = 0;

void enqueue(struct Order o) {
    queue[rear++] = o;
}
```

```
struct Order dequeue() {
   return queue[front++];
}

// ================= HEAP =================
struct Order heap[HEAP_SIZE];
int heapSize = 0;

void swap(struct Order *a, struct Order *b) {
   struct Order temp = *a;
   *a = *b;
   *b = temp;
}

void heapInsert(struct Order o) {
   heap[heapSize] = o;
   int i = heapSize++;

   while(i > 0 && heap[(i-1)/2].food.price < heap[i].food.price) {
      swap(&heap[(i-1)/2], &heap[i]);
      i = (i-1)/2;
   }
}

struct Order heapExtract() {
   struct Order root = heap[0];
   heap[0] = heap[--heapSize];

   int i = 0;
   while(2*i+1 < heapSize) {
      int largest = i;
      int l = 2*i+1;
      int r = 2*i+2;

      if(heap[l].food.price > heap[largest].food.price)
         largest = l;

      if(r < heapSize && heap[r].food.price > heap[largest].food.price)
         largest = r;

      if(largest == i) break;

      swap(&heap[i], &heap[largest]);
      i = largest;
```

```
    }

    return root;
}

// ================= GLOBAL MENU =================
struct Food menu[MAX_MENU];
int menuCount = 0;

struct BST* root = NULL;

// ================= ADD FOOD =================
void addFood(int id, char name[], float price) {
    struct Food f;
    f.id = id;
    strcpy(f.name, name);
    f.price = price;

    menu[menuCount++] = f;

    root = insertBST(root, f);
    insertHash(f);
}

// ================= SHOW MENU =================
void showMenu() {
    printf("\n===== MENU =====\n");
    printf("ID\tName\tPrice\n");
    for(int i=0;i<menuCount;i++) {
        printf("%d\t%s\t%.2f\n", menu[i].id, menu[i].name, menu[i].price);
    }
}

// ================= ORDER LIST =================
void showOrders() {
    printf("\n===== ORDER QUEUE =====\n");
    for(int i=front;i<rear;i++) {
        printf("%s x%d\n", queue[i].food.name, queue[i].qty);
    }
}

// ================= TOTAL BILL =================
void calculateBill() {
    float total = 0;
```

```
    for(int i=front;i<rear;i++) {
        total += queue[i].food.price * queue[i].qty;
    }
    printf("\nTOTAL BILL: %.2f\n", total);
}

// ================= INIT =================
void init() {
    addFood(1,"Burger",150);
    addFood(2,"Pizza",500);
    addFood(3,"Pasta",250);
    addFood(4,"Chicken",300);
    addFood(5,"Coffee",100);
}

// ================= MAIN =================
int main() {
    init();

    int choice, id, qty;

    while(1) {
        printf("\n==== FOOD SYSTEM ====\n");
        printf("1. Show Menu\n");
        printf("2. Search (BST)\n");
        printf("3. Search (Hash)\n");
        printf("4. Place Order\n");
        printf("5. Show Orders\n");
        printf("6. Serve Priority Order\n");
        printf("7. Calculate Bill\n");
        printf("0. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        if(choice == 0) break;

        switch(choice) {

        case 1:
            showMenu();
            break;

        case 2:
```

```
    printf("Enter ID: ");
    scanf("%d", &id);
    {
        struct Food f = searchBST(root, id);
        if(f.id == -1) printf("Not Found\n");
        else printf("%s %.2f\n", f.name, f.price);
    }
    break;

case 3:
    printf("Enter ID: ");
    scanf("%d", &id);
    {
        struct Food f = searchHash(id);
        if(f.id == -1) printf("Not Found\n");
        else printf("%s %.2f\n", f.name, f.price);
    }
    break;

case 4:
    printf("Enter Food ID & Qty: ");
    scanf("%d %d", &id, &qty);

    {
        struct Food f = searchBST(root, id);
        if(f.id == -1) {
            printf("Invalid ID\n");
            break;
        }

        struct Order o = {f, qty};
        enqueue(o);
        heapInsert(o);

        printf("Order Added: %s x%d\n", f.name, qty);
    }
    break;

case 5:
    showOrders();
    break;

case 6:
    if(heapSize == 0) {
```

```
            printf("No priority orders\n");
            break;
        }
        {
            struct Order o = heapExtract();
            printf("Serving: %s x%d\n", o.food.name, o.qty);
        }
        break;

    case 7:
        calculateBill();
        break;

    default:
        printf("Invalid choice\n");
    }
  }

  return 0;
}
```